(CE7456 Lecture Notes 8) Distribution-Aware Data Structuring (a.k.a. Learned Indexes)

Yufei Tao

So far all the complexities in our algorithmic analysis are claimed in the *worst case*. For example, by saying that the binary search tree (BST) consumes $O(n)$ space and answers a predecessor search in $O(\log n)$ time, we require these complexities to hold on *every* set of n integers and *every* query. Today, we will switch to a different form of analysis for deriving *average complexities*, which hold in expectation when data and query follow specific distributions.

We still use predecessor search as the representative example: the input (i.e., the dataset) is a collection S of n integers; given an integer q, a query returns the the largest element $e \in S$ satisfying $e \leq q$, or declares that such an element does not exist. All the integers — including those in S and the query value q — are assumed to fall in the domain $[U]$ (for an integer $x > 0$, the notation $[x]$ represents the set $\{1, 2, ..., x\}$) where U is a (large) integer. W.l.o.g., we consider that U is a multiple of n (if not, increase U to make the property hold).

Let $\mathcal{D}$ be a probabilistic distribution over $[U]$. We consider that

- the n integers of S are drawn independently from $\mathcal{D}$ (namely, S is drawn from $\mathcal{D}^n$);
- the query value q is drawn from $\mathcal{D}$ and is independent from S.

Let $Time_S(Q)$ be the time of an algorithm $\mathcal{A}$ in answering a query with search value Q on a dataset S. Note that both S and Q are random variables and, hence, so is $Time_S(Q)$. We define the *average time complexity* of $\mathcal{A}$ over $\mathcal{D}$ as

$$\mathop{\mathbf{E}}_{Q,S}[Time_S(Q)].$$

The worst-case $O(\log n)$ guarantee of BST directly implies an average query time of $O(\log n)$. The challenge is to find a data structure with a lower average query time on "easy" distributions $\mathcal{D}$.

1 Structure

Define

$$\begin{aligned} a &= \min_{e \in S} e \\ b &= \max_{e \in S} e. \end{aligned}$$

We consider $a < b$ (the problem is trivial otherwise; think: why?). Use an array A to store the elements of S in ascending order. Divide the range $[a, b]$ into n intervals with equal length; specifically, the i-th interval ($i \in [n]$) is

$$I_i = \begin{cases} \left[a + (b-a) \cdot \frac{i-1}{n}, a + (b-a) \cdot \frac{i}{n}\right) & \text{if } i < n \\ \left[a + (b-a) \cdot \frac{i-1}{n}, a + (b-a) \cdot \frac{i}{n}\right] & \text{if } i = n. \end{cases} \quad (1)$$

For each $i \in [n]$, define

$$n_i \quad = \quad |S \cap I_i|. \tag{2}$$

and a function $\hat{r}_S : [U] \to [n]$ as:

$$\hat{r}_S(q) \quad = \quad \begin{cases} 0 & \text{if } q < a \\ \frac{n_i}{2} + \sum_{k=1}^{i-1} n_i & \text{if } q \in I_i \text{ for some } i \in [n] \\ n & \text{if } q > b \end{cases} \tag{3}$$

Our data structure consists of (i) the values a, b and n, (ii) the sorted array A, and (iii) the function $\hat{r}_S$. In particular, we store $\hat{r}_S$ in an array of size n such that, for any $q \in I_i$ with $i \in [n]$, we can return $\hat{r}_S(q)$ in constant time. The total space consumption is $O(n)$ in the worst case.

2 Query

Let q be the query value of predecessor search. Define

$$rank_S(q) \quad = \quad \begin{cases} 0 & \text{if } q < a \\ \text{the number of elements in } S \text{ at most } q & \text{if } a \leq q \leq b \\ n & \text{if } q > b \end{cases} \tag{4}$$

Note that when $q > a$, the predecessor of q is $A[rank_S(q)]$.

We answer the query as follows. If $q < a$, return "absence". Consider now $q \geq a$. We check whether $A[\hat{r}_S(q)] = q$ — this takes constant time — and return $\hat{r}_S(q)$ if so. If $q < A[\hat{r}_S(q)]$, look for q using exponential search in the subarray from $A[1]$ to $A[\hat{r}_S(q)]$ (see Appendix A for an introduction to exponential search); otherwise, look for q using exponential search in the subarray from $A[\hat{r}_S(q)]$ to $A[n]$.

If

$$\Delta_S(q) \quad = \quad |rank_S(q) - \hat{r}_S(q)| \tag{5}$$

the query time is bounded by $O(\log \Delta_S(q))$.

We mention a proposition here that will be useful later:

Proposition 1. *If $q \in I_i$ (for some $i \in [n]$), then $\Delta_S(q) \leq n_i/2$.*

The proof is easy and left to you.

3 Analysis

Define f as the probability density function of $\mathcal{D}$, namely, $Pr[X = x] = f(x)$ for a random variable X drawn from $\mathcal{D}$. We will prove:

Theorem 1. *The average query time of the algorithm in Section 2 is*

$$O\Big(\log\Big(U \cdot \sum_{q \in [U]} (f(q))^2\Big)\Big).$$

For example, when $\mathcal{D}$ is uniform over $[U]$, we have $f(x) = 1/U$ for all $x \in [U]$, in which case the average query time is $O(\log(U \cdot (U \cdot 1/U^2))) = O(1)$. More generally, the average query complexity is $O(\log t)$ if $f(x) \leq t/U$ for all $x \in [U]$ — for t is sub-polynomial in n, the structure strictly improves the BST (in terms of average running time).

3.1 Domain-Partitioning Intervals

To facilitate our analysis, we partition the domain $[U]$ into n intervals of equal length; specifically, for each $j \in [n]$, the j-th interval is

$$J_j = \begin{cases} \left[1 + (U-1) \cdot \frac{j-1}{n}, 1 + (U-1) \cdot \frac{j}{n}\right) & \text{if } j < n \\ \left[1 + (U-1) \cdot \frac{j-1}{n}, 1 + (U-1) \cdot \frac{j}{n}\right] & \text{if } j = n. \end{cases} \tag{6}$$

Specially, define

$$J_0 = J_{n+1} = \emptyset.$$

For each $j \in \{0, 1, ..., n+1\}$, set

$$m_j = |S \cap J_j|. \tag{7}$$

Each interval J_j ($j \in [n]$) has a length $(U-1)/n$, which is at least that of an interval I_i ($i \in [n]$) defined in (1). The proposition below follows immediately:

Proposition 2. *Each interval I_i ($i \leq [n]$) defined in* (1) *is covered by $J_j \cup J_{j+1}$ for some $j \in [n-1]$.*

Corollary 2. *Fix an arbitrary $j \in [n]$. If an interval I_i ($i \leq [n]$) intersects with J_j, then I_i is covered by $J_{j-1} \cup J_j \cup J_{j+1}$.*

The proof is easy and left to you.

3.2 A Core Lemma

Define for each $j \in [n]$

$$p_j = \mathbf{Pr}_{X \sim \mathcal{D}}[X \in J_j] = \sum_{q \in J_j} f(q). \tag{8}$$

This section serves as the proof of the lemma below:

Lemma 3. $\mathbf{E}_{Q,S}[\Delta_S(Q)] = O(n) \cdot \sum_{j=1}^{n} p_j^2$.

Proof of Lemma 3. The total law of expectation tells us:

$$\begin{aligned} \mathop{\mathbf{E}}_{Q,S}[\Delta_S(Q)] &= \mathop{\mathbf{E}}_{S}\left[\mathop{\mathbf{E}}_{Q|S}\left[\Delta_S(Q)|S\right]\right] \\ \text{(by independence of } Q \text{ and } S) &= \mathop{\mathbf{E}}_{S}\left[\mathop{\mathbf{E}}_{Q}\left[\Delta_S(Q)|S\right]\right]. \end{aligned} \tag{9}$$

Regarding $\mathbf{E}_Q[\Delta_S(Q)|S]$, we can derive:

$$\begin{aligned} \mathop{\mathbf{E}}_{Q}[\Delta_S(Q)|S] &= \sum_{q \in [U]} \Delta_S(q) \cdot f(q) \\ &= \sum_{i \in [n]} \sum_{q \in I_i} \Delta_S(q) \cdot f(q) \end{aligned}$$

$$\text{(by Proposition 1)} \quad \leq \quad \sum_{i=1}^{n} \frac{n_i}{2} \sum_{q \in I_i} f(q). \tag{10}$$

Define for each $i, j \in [n]$:

$$n_{i,j} = |S \cap I_i \cap J_j|.$$

Thus

$$(10) = \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} n_{i,j} \sum_{q \in I_i} f(q) = \frac{1}{2} \sum_{j=1}^{n} \sum_{i=1}^{n} n_{i,j} \sum_{q \in I_i} f(q). \tag{11}$$

For a specific $j \in [n]$, we have

$$\begin{aligned}
\sum_{i=1}^{n} n_{i,j} \sum_{q \in I_i} f(q) &= \sum_{i \in [n]: I_i \cap J_j \neq \emptyset} n_{i,j} \sum_{q \in I_i} f(q) \\
\text{(by Corollary 2)} \quad &\leq \sum_{i \in [n]: I_i \cap J_j \neq \emptyset} n_{i,j} \sum_{q \in J_{j-1} \cup J_j \cup J_{j+1}} f(q) \\
&= m_j \cdot (p_{j-1} + p_j + p_{j+1}).
\end{aligned}$$

where m_j and p_j are defined in (7) and (8), respectively. Plugging the above into (11) and then into (10) yields

$$\mathop{\mathbf{E}}_{Q}[\Delta_S(Q)|S] = \frac{1}{2} \sum_{j=1}^{n} m_j \cdot (p_{j-1} + p_j + p_{j+1})$$

which, together with (9), gives

$$\begin{aligned}
\mathop{\mathbf{E}}_{Q,S}[\Delta_S(Q)|S] = \mathop{\mathbf{E}}_{S}[\mathop{\mathbf{E}}_{Q}[\Delta_S(Q)|S]] &= \frac{1}{2} \cdot \mathop{\mathbf{E}}_{S} \Big[\sum_{j=1}^{n} m_j \cdot (p_{j-1} + p_j + p_{j+1}) \Big] \\
&= \frac{1}{2} \cdot \sum_{j=1}^{n} \mathop{\mathbf{E}}_{S}[m_j] \cdot (p_{j-1} + p_j + p_{j+1}) \\
&= \frac{1}{2} \cdot \sum_{j=1}^{n} n p_j \cdot (p_{j-1} + p_j + p_{j+1}) \\
&= \frac{n}{2} \cdot \sum_{j=1}^{n} p_j p_{j-1} + p_j^2 + p_j p_{j+1}. \qquad (12)
\end{aligned}$$

Proposition 3. $\sum_{j=1}^{n} p_j p_{j-1} = O(\sum_{j=1}^{n} p_j^2)$ *and* $\sum_{j=1}^{n} p_j p_{j+1} = O(\sum_{j=1}^{n} p_j^2)$.

The proof is left to you as an exercise. Lemma 3 now follows by combining the above proposition and (12).

3.3 Proof of Theorem 1

By the Cauchy-Schwarz inequality[1], we have for each $j \in [n]$:

$$p_j^2 = \Big(\sum_{q \in J_j} f(q)\Big)^2 \leq \Big(\sum_{q \in J_j} 1\Big) \cdot \Big(\sum_{q \in J_j} (f(q))^2\Big) \leq O\Big(\frac{U}{n}\Big) \cdot \sum_{q \in J_j} (f(q))^2$$

where the last step used the fact that J_j contains $O(U/n)$ integers. Plugging the above into Lemma 3 yields:

$$\begin{aligned} \mathop{\mathbf{E}}_{Q,S}[\Delta_S(Q)] &= O(n) \cdot \sum_{j=1}^{n} O\Big(\frac{U}{n}\Big) \cdot \sum_{q \in J_j} (f(q))^2 \\ &= O(U) \cdot \sum_{j=1}^{n} \sum_{q \in J_j} (f(q))^2 \\ &= O(U) \cdot \sum_{q \in [U]} (f(q))^2. \end{aligned} \tag{13}$$

The average running time of the query algorithm in Section 2 is bounded by

$$O\Big(\mathop{\mathbf{E}}_{Q,S} \log \Delta_S(q)\Big) \leq O\Big(\log \mathop{\mathbf{E}}_{Q,S} \Delta_S(q)\Big)$$

where the inequality is due to Jensen's inequality. The claim in Theorem 1 now follows from the above and (13).

4 Remarks

The structure and analysis presented were developed in [1].

References

[1] Luis Alberto Croquevielle, Guang Yang, Liang Liang, Ali Hadian, and Thomas Heinis. Beyond logarithmic bounds: Querying in constant expected time with learned indexes. In *Proceedings of International Conference on Database Theory (ICDT)*, pages 19:1–19:21, 2025.

A Exponential Search

Let A be a sorted array of n elements (in non-decreasing order), and let q be a target value. Exponential search decides whether q appears in A — if so, return its index — as follows. If $q < A[1]$, return "absence". Otherwise, identify the smallest $h \in [n]$ such that

- h is a power of 2 or $h = n$, and
- $A[n] \geq q$

The value q — if it exists in A — must lie in the subarray from $A[\lceil h/2 \rceil]$ to $A[h]$. We then perform binary search to look for q in the subarray.

Let Δ be the number of elements in A that are at most q. Clearly, $h \leq 2\Delta$. It is rudimentary to show that the running time is $O(\log \Delta)$.

[1] $(\sum_{i=1}^{n} x_i y_i)^2 \leq \sum_{i=1}^{n} x_i^2 \cdot \sum_{i=1}^{n} y_i^2$.